

TUGAS PRAKTIKUM 3

SISTEM OPERASI



MUHAMMAD MAULANA NUR SULAIMAN

2410651033

UNIVERITAS MUHAMMDIYAH JEMBER

FAKULTAS TEKNIK

PROGRAM STUDI TEKNIK INFORMATIKA

2025

Tugas 1: Kalkulator Paralel

Buat program yang:

1. Menerima input 2 angka dari user
2. Menghitung penjumlahan, pengurangan, perkalian, dan pembagian
3. Setiap operasi dikerjakan oleh thread berbeda
4. Tampilkan semua hasil setelah semua thread selesai

a. Buat file program baru:

```
nano kalkulator_paralel.c
```

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
// Struktur untuk menyimpan data input dan hasil
```

```
typedef struct {
```

```
    double a, b;
```

```
    double hasil;
```

```
} Data;
```

```
// Fungsi untuk tiap operasi
```

```
void* tambah(void* arg) {
```

```
    Data* data = (Data*)arg;
```

```
    data->hasil = data->a + data->b;
```

```
    pthread_exit(NULL);
```

```
}
```

```
void* kurang(void* arg) {
```

```
    Data* data = (Data*)arg;
```

```
    data->hasil = data->a - data->b;
```

```
    pthread_exit(NULL);
```

```
}
```

```
void* kali(void* arg) {
```

```
    Data* data = (Data*)arg;
```

```
    data->hasil = data->a * data->b;
```

```
    pthread_exit(NULL);
```

```
}
```

```
void* bagi(void* arg) {
```

```
    Data* data = (Data*)arg;
```

```
    if (data->b != 0)
```

```
        data->hasil = data->a / data->b;
```

```
    else
```

```
        data->hasil = 0; // hindari pembagian nol
```

```
    pthread_exit(NULL);
```

```
}
```

```
int main() {

    double x, y;

    pthread_t t1, t2, t3, t4;

    Data dTambah, dKurang, dKali, dBagi;


    printf("=== KALKULATOR PARALEL ===\n");

    printf("Masukkan angka pertama : ");

    scanf("%lf", &x);

    printf("Masukkan angka kedua  : ");

    scanf("%lf", &y);


    // Set data untuk masing-masing thread

    dTambah.a = dKurang.a = dKali.a = dBagi.a = x;

    dTambah.b = dKurang.b = dKali.b = dBagi.b = y;


    // Buat thread untuk setiap operasi

    pthread_create(&t1, NULL, tambah, &dTambah);

    pthread_create(&t2, NULL, kurang, &dKurang);

    pthread_create(&t3, NULL, kali, &dKali);

    pthread_create(&t4, NULL, bagi, &dBagi);
```

```

// Tunggu semua thread selesai

pthread_join(t1, NULL);

pthread_join(t2, NULL);

pthread_join(t3, NULL);

pthread_join(t4, NULL);


// Tampilkan hasil setelah semua selesai

printf("\n=== HASIL PERHITUNGAN ===\n");

printf("Penjumlahan : %.2lf\n", dTambah.hasil);

printf("Pengurangan : %.2lf\n", dKurang.hasil);

printf("Perkalian   : %.2lf\n", dKali.hasil);

printf("Pembagian    : %.2lf\n", dBagi.hasil);


return 0;

}

```

Program ini merupakan sebuah kalkulator parallel yang memanfaatkan multithread dengan menggunakan Pustaka pthread pada Bahasa C.

Program ini meminta dua angka dari pengguna (x dan y). Setiap jenis operasi matematika penjumlahan, pengurangan, perkalian, dan pembagian dan dijalankan oleh thread yang berbeda secara bersamaan.

Struktur data dipakai untuk menyimpan kedua angka masukan (a, b) serta hasil perhitungan. Fungsi untuk menjumlahkan, mengurangi, mengalikan, dan membagi masing-masing melaksanakan operasi matematika serta menyimpan hasilnya pada

struktur data. Setelah semua thread menyelesaikan tugasnya (pthread_join), hasil dari setiap operasi ditampilkan di layar.

b. Compile dan jalan

```
gcc kalkulator_paralel.c -o kalkulator_paralel -lpthread
./kalkulator_paralel
```

```
alangenteng@LAPTOP-CV1QG677:~$ nano kalkulator_paralel.c
alangenteng@LAPTOP-CV1QG677:~$ gcc kalkulator_paralel.c -o kalkulator_paralel -lpthread
alangenteng@LAPTOP-CV1QG677:~$ ./kalkulator_paralel
=== KALKULATOR PARALEL ===
Masukkan angka pertama : 20
Masukkan angka kedua  : 17

=== HASIL PERHITUNGAN ===
Penjumlahan : 37.00
Pengurangan : 3.00
Perkalian   : 340.00
Pembagian   : 1.18
alangenteng@LAPTOP-CV1QG677:~$
```

Tugas 2: File Processing

Buat program yang:

1. Membaca file teks
2. Menghitung jumlah kata menggunakan 2 thread (bagi file jadi 2 bagian)
3. Gabungkan hasilnya
4. Bandingkan waktu eksekusi dengan versi single-thread

a. Buat file program baru:

```
nano file_processing.c
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <string.h>
```

```
#include <time.h>
```

```
typedef struct {
```

```
    char *text;
```

```
    long start;
```

```
    long end;
```

```
    int count;
```

```
} ThreadData;
```

```
// Fungsi untuk menghitung kata di bagian tertentu
```

```
void* hitung_kata(void* arg) {
```

```
    ThreadData* data = (ThreadData*) arg;
```

```
    int inWord = 0;
```

```
    int count = 0;
```

```
    for (long i = data->start; i < data->end; i++) {
```

```
        char c = data->text[i];
```

```
        if (c == ' ' || c == '\n' || c == '\t' || c == '\r' || c == '\0') {
```

```
            if (inWord) {
```

```
                count++;
```

```

        inWord = 0;

    }

    } else {

        inWord = 1;

    }

}

data->count = count;

pthread_exit(NULL);

}

```

// Fungsi untuk membaca isi file

```

char* baca_file(const char* filename, long* ukuran) {

    FILE* file = fopen(filename, "r");

    if (!file) {

        printf("Gagal membuka file %s\n", filename);

        exit(1);

    }

    fseek(file, 0, SEEK_END);

    *ukuran = ftell(file);

    rewind(file);

    char* buffer = (char*)malloc(*ukuran + 1);

```



```
fread(buffer, 1, *ukuran, file);

buffer[*ukuran] = '\0';

fclose(file);

return buffer;

}
```

// Hitung kata secara single-thread

```
int hitung_kata_single(char* text, long size) {

    int inWord = 0;

    int count = 0;

    for (long i = 0; i < size; i++) {

        char c = text[i];

        if (c == ' ' || c == '\n' || c == '\t' || c == '\r' || c == '\0') {

            if (inWord) {

                count++;

                inWord = 0;

            }

        } else {

            inWord = 1;

        }

    }

    return count;

}
```

```
}
```

```
int main() {
```

```
    char filename[100];
```

```
    printf("=== FILE PROCESSING (Word Count) ===\n");
```

```
    printf("Masukkan nama file teks: ");
```

```
    scanf("%s", filename);
```

```
    long size;
```

```
    char* text = baca_file(filename, &size);
```

```
    printf("\nUkuran file: %ld byte\n", size);
```

```
    // === SINGLE THREAD ===
```

```
    clock_t start_single = clock();
```

```
    int total_single = hitung_kata_single(text, size);
```

```
    clock_t end_single = clock();
```

```
    double waktu_single = (double)(end_single - start_single) /  
CLOCKS_PER_SEC;
```

```
    // === MULTI THREAD (2 THREAD) ===
```

```
    pthread_t t1, t2;
```

```
    ThreadData d1, d2;
```

```

long mid = size / 2;

d1.text = d2.text = text;

d1.start = 0;

d1.end = mid;

d2.start = mid;

d2.end = size;

clock_t start_multi = clock();

pthread_create(&t1, NULL, hitung_kata, &d1);

pthread_create(&t2, NULL, hitung_kata, &d2);

pthread_join(t1, NULL);

pthread_join(t2, NULL);


int total_multi = d1.count + d2.count;

clock_t end_multi = clock();

double waktu_multi = (double)(end_multi - start_multi) /
CLOCKS_PER_SEC;


// === HASIL ===

printf("\n=== HASIL PERHITUNGAN ===\n");

printf("Jumlah kata (Single Thread) : %d\n", total_single);

printf("Jumlah kata (Multi Thread) : %d\n", total_multi);

```

```

printf("\nWaktu eksekusi (Single Thread) : %.6f detik\n", waktu_single);

printf("Waktu eksekusi (Multi Thread) : %.6f detik\n", waktu_multi);

free(text);

return 0;

}

```

Program yang ditulis diatas merupakan syntax untuk menghitung total kata (Word Count) menggunakan satu thread dan beberapa thread sebagai cara uuntuk membandingkan kecepatan operasional.

Program ini meminta pengguna untuk memasukkan nama file teks dan kemudian membaca seluruh konten ke dalam memori menggunakan fungsi `baca_file()`.

Fungsi `hitung_file_single()` berfungsi untuk menghitung total kata secara berurutan (1 thread). Fungsi `hitung_kata()` dijalankan oleh dua thread (multi-thread) yang masing-masing mengolah setengah dari isi file. Setelah thread kedua selesai, hasil perhitungan dari tiap thread dikumpulkan.

Kemudian, program menampilkan total jumlah kata yang diperoleh melalui single thread dan multi thread, beserta waktu eksekusinya untuk melakukan perbandingan.

b. Compile dan jalan

```

gcc file_processing.c -o file_processing -lpthread
./file_processing

```

```

alangenteng@LAPTOP-CV1QG677:~$ ./file_processing
=== FILE PROCESSING (Word Count) ===
Masukkan nama file teks: k
Gagal membuka file k
alangenteng@LAPTOP-CV1QG677:~$ nano contoh.txt
alangenteng@LAPTOP-CV1QG677:~$ ./file_processing
=== FILE PROCESSING (Word Count) ===
Masukkan nama file teks: contoh.txt

Ukuran file: 747 byte

=== HASIL PERHITUNGAN ===
Jumlah kata (Single Thread) : 84
Jumlah kata (Multi Thread) : 84

Waktu eksekusi (Single Thread) : 0.000003 detik
Waktu eksekusi (Multi Thread) : 0.001861 detik
alangenteng@LAPTOP-CV1QG677:~$ _

```

Tugas 3: Producer-Consumer

Buat simulasi producer-consumer dengan:

1. Thread producer yang menghasilkan angka random
2. Thread consumer yang memproses angka tersebut
3. Gunakan mutex untuk sinkronisasi
4. Buffer maksimal 10 item

a. Buat file program baru:

```

nano prouducer_consumer

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h> // untuk sleep()

```

```

#include <time.h>

#define BUFFER_SIZE 10

int buffer[BUFFER_SIZE];
int count = 0; // jumlah item dalam buffer

pthread_mutex_t mutex;
pthread_cond_t notFull;
pthread_cond_t notEmpty;

// Fungsi untuk menambahkan item ke buffer
void produce(int item) {
    pthread_mutex_lock(&mutex);

    // Tunggu jika buffer penuh
    while (count == BUFFER_SIZE) {
        printf("[Producer] Buffer penuh, menunggu...\n");
        pthread_cond_wait(&notFull, &mutex);
    }

    buffer[count] = item;
    count++;
    printf("[Producer] Menghasilkan item: %d (buffer terisi %d/%d)\n",
item, count, BUFFER_SIZE);

    pthread_cond_signal(&notEmpty); // beri sinyal ke consumer
    pthread_mutex_unlock(&mutex);
}

// Fungsi untuk mengambil item dari buffer

```

```

int consume() {
    pthread_mutex_lock(&mutex);

    // Tunggu jika buffer kosong
    while (count == 0) {
        printf("[Consumer] Buffer kosong, menunggu...\n");
        pthread_cond_wait(&notEmpty, &mutex);
    }

    int item = buffer[count - 1];
    count--;
    printf("[Consumer] Mengonsumsi item: %d (buffer terisi %d/%d)\n",
item, count, BUFFER_SIZE);

    pthread_cond_signal(&notFull); // beri sinyal ke producer
    pthread_mutex_unlock(&mutex);

    return item;
}

// Thread producer
void* producer(void* arg) {
    while (1) {
        int item = rand() % 100; // angka random 0–99
        produce(item);
        sleep(1); // simulasi waktu produksi
    }
}

// Thread consumer
void* consumer(void* arg) {

```

```

while (1) {
    int item = consume();
    // simulasi waktu proses item
    printf("  -> [Consumer] Memproses item %d...\n", item);
    sleep(2);
}
}

int main() {
    pthread_t prodThread, consThread;
    srand(time(NULL));

    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&notFull, NULL);
    pthread_cond_init(&notEmpty, NULL);

    printf("=== SIMULASI PRODUCER-CONSUMER ===\n");

    // Buat thread producer dan consumer
    pthread_create(&prodThread, NULL, producer, NULL);
    pthread_create(&consThread, NULL, consumer, NULL);

    // Jalankan terus (CTRL + C untuk berhenti)
    pthread_join(prodThread, NULL);
    pthread_join(consThread, NULL);

    // Bersihkan resource
    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&notFull);
    pthread_cond_destroy(&notEmpty);

```



```
    return 0;
}
```

Program ini adalah sebuah simulasi dari konsep producer_consumer dengan penerapan multithreading dalam Bahasa C, menggunakan sinkronisasi melalui mutex dan condition variable.

Program ini memiliki buffer yang memiliki ukuran tetap (10 slot) yang digunakan secara bersama oleh dua thread:

Producer – menciptakan data acak (0-99) dan menambahkannya ke dalam buffer.

Consumer – mengambil data dari buffer untuk di proses lebih lanjut.

Mutex (pthread_mutex_t) diterapkan untuk melindungi akses buffer sehingga tidak terjadi konflik antar thread.

Condition variable (pthread_cond_t):

notFull berfungsi untuk menghentikan producer agar menunggu Ketika buufer sudah penuh.

notEmpty membuat consumer menunggu ketika buffer tidak memiliki item atau ketika buffer kosong.

Thread producer menghasilkan item setiap 1 detik, sementara consumer mengambil item setiap 2 detik.

Program akan terus berjalan tanpa henti hingga dihentikan secara manual (CTRL + C).

b. Compile dan jalan

```
gcc producer_consumer.c -o producer_consumer -lpthread
./producer_consumer
```

```
alangenteng@LAPTOP-CV1QG677:~$ nano producer_consumer.c
alangenteng@LAPTOP-CV1QG677:~$ gcc producer_consumer.c -o producer_consumer -lpthread
alangenteng@LAPTOP-CV1QG677:~$ ./producer_consumer
=== SIMULASI PRODUCER-CONSUMER ===
[Producer] Menghasilkan item: 25 (buffer terisi 1/10)
[Consumer] Mengonsumsi item: 25 (buffer terisi 0/10)
-> [Consumer] Memproses item 25...
[Producer] Menghasilkan item: 4 (buffer terisi 1/10)
[Consumer] Mengonsumsi item: 4 (buffer terisi 0/10)
-> [Consumer] Memproses item 4...
[Producer] Menghasilkan item: 37 (buffer terisi 1/10)
[Producer] Menghasilkan item: 51 (buffer terisi 2/10)
[Consumer] Mengonsumsi item: 51 (buffer terisi 1/10)
-> [Consumer] Memproses item 51...
[Producer] Menghasilkan item: 53 (buffer terisi 2/10)
[Producer] Menghasilkan item: 11 (buffer terisi 3/10)
[Consumer] Mengonsumsi item: 11 (buffer terisi 2/10)
-> [Consumer] Memproses item 11...
[Producer] Menghasilkan item: 18 (buffer terisi 3/10)
[Producer] Menghasilkan item: 32 (buffer terisi 4/10)
[Consumer] Mengonsumsi item: 32 (buffer terisi 3/10)
-> [Consumer] Memproses item 32...
[Producer] Menghasilkan item: 74 (buffer terisi 4/10)
[Producer] Menghasilkan item: 31 (buffer terisi 5/10)
[Consumer] Mengonsumsi item: 31 (buffer terisi 4/10)
-> [Consumer] Memproses item 31...
[Producer] Menghasilkan item: 42 (buffer terisi 5/10)
[Producer] Menghasilkan item: 29 (buffer terisi 6/10)
[Consumer] Mengonsumsi item: 29 (buffer terisi 5/10)
-> [Consumer] Memproses item 29...
[Producer] Menghasilkan item: 92 (buffer terisi 6/10)
[Producer] Menghasilkan item: 15 (buffer terisi 7/10)
[Consumer] Mengonsumsi item: 15 (buffer terisi 6/10)
-> [Consumer] Memproses item 15...
[Producer] Menghasilkan item: 22 (buffer terisi 7/10)
[Producer] Menghasilkan item: 31 (buffer terisi 8/10)
[Consumer] Mengonsumsi item: 31 (buffer terisi 7/10)
-> [Consumer] Memproses item 31...
[Producer] Menghasilkan item: 16 (buffer terisi 8/10)
[Producer] Menghasilkan item: 11 (buffer terisi 9/10)
[Consumer] Mengonsumsi item: 11 (buffer terisi 8/10)
-> [Consumer] Memproses item 11...
[Producer] Menghasilkan item: 10 (buffer terisi 9/10)
[Producer] Menghasilkan item: 22 (buffer terisi 10/10)
[Consumer] Mengonsumsi item: 22 (buffer terisi 9/10)
```